CSS4005 — Cloud Infrastructure Construction

Lecture 8: AWS Architecture

Nauruzbayeva Farikha · Lesson 8

Recap — Lesson 7 (Amazon RDS)

- What does RDS manage that a database on EC2 doesn't? (OS and engine patching, automated backups and point-in-time restore, Multi-AZ standby and failover)
- Why is SG-DB's source SG-APP instead of an IP range? (it trusts the app servers wherever they are, and nothing else in the subnet)
- You restore a snapshot. What changes for the app? (a restore creates a **new** instance with a **new** endpoint — the app must be pointed at it)
- Multi-AZ or read replica for "survive an AZ outage"? (Multi-AZ: synchronous standby, automatic failover, same endpoint)

Today’s agenda

- ☐ The AWS Well-Architected Framework: six pillars
- ☐ Reading and drawing architecture diagrams
- ☐ Building a three-tier architecture from parts you already know
- ☐ Single points of failure — and how to remove them
- ☐ A design review checklist → today’s practice

The scenario

You know how to launch an EC2 instance, build a VPC, store files in S3, lock things down with IAM and run a database on RDS.

Your manager asks: "Draw me how our app runs on AWS, and tell me what happens if a whole data center goes down." Seven separate skills — one diagram, one answer.

Today: put the pieces together, and learn to defend a design.

Block 1

The AWS Well-Architected Framework

What it is

- AWS's set of **design principles and questions** for judging whether a workload is built well.
- Organised into **six pillars**. A good design makes conscious trade-offs between them — it rarely maximises all six.
- Free to read: the Well-Architected Framework whitepaper, and the Well-Architected Tool in the AWS console for structured reviews.

You already applied most of it: Multi-AZ, least privilege, deleting unused resources — today we give those habits names.

The six pillars

Pillar	The question it asks
Operational excellence	Can we run, monitor and change it safely?
Security	Is data and access protected at every layer?
Reliability	Does it keep working — and recover — when parts fail?
Performance efficiency	Are we using the right resources, and adapting as demand changes?
Cost optimization	Are we paying only for what delivers value?
Sustainability	Are we minimising the environmental impact of running it?

One example per pillar

- **Operational excellence:** bootstrap servers with user data (or scripts) instead of clicking — repeatable and reviewable.
- **Security:** instance profile instead of access keys; SG-DB only from SG-APP; Block Public Access on S3.
- **Reliability:** two AZs for the app tier; RDS Multi-AZ; automated backups with a tested restore.
- **Performance efficiency:** pick the instance type and EBS volume type for the workload; serve static files from S3.
- **Cost optimization:** stop or delete what's idle; lifecycle rules to cheaper S3 classes; right-size instances.
- **Sustainability:** right-size and turn off unused resources — less waste is also less energy.

Quick check

You turn on RDS Multi-AZ. Which pillar does that serve, and what does it cost you?

Reliability. The trade-off is cost: Multi-AZ runs a standby instance, so you pay for roughly a second DB instance.

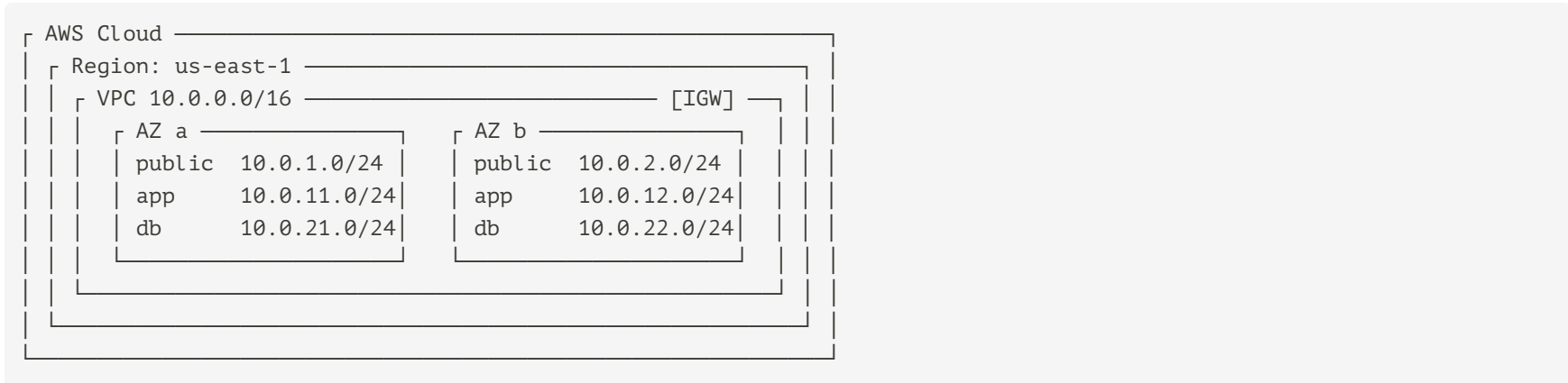
Block 2

Reading and drawing architecture diagrams

Diagram conventions

- **Nested boxes** show scope, outside in: AWS Cloud → **Region** → **VPC** → **Availability Zone** → **Subnet**.
- Public and private subnets are usually coloured differently (AWS's set uses green for public, blue for private).
- **Service icons** come from the official AWS Architecture Icons set — available in diagrams.net (draw.io) under "AWS".
- **Arrows** show request flow; label them with port/protocol when it matters (e.g. `HTTPS 443`, `5432`).
- Things outside the VPC (S3, IAM) sit outside the VPC box, inside the region or cloud.

The skeleton every diagram starts with



Two AZs, three subnet tiers in each — before a single server is drawn.

What a reviewer looks for in a diagram

- Is every subnet clearly **public or private** — and does that match its route table?
- Is anything that shouldn't be reachable from the internet **in a private subnet**?
- Does every tier exist in **more than one AZ**?
- Are the **arrows and ports** consistent with the security groups?
- Where is **data stored**, and how is it backed up?

Block 3

Building a three-tier architecture

Step 1 — Network

- One VPC, **two AZs**.
- In each AZ: a **public** subnet, an **app** (private) subnet, a **DB** (private) subnet.
- An **Internet Gateway**; public route table `0.0.0.0/0 → IGW`.
- A **NAT gateway** in a public subnet, so private instances can download updates without being reachable from the internet.

For high availability, production designs often put one NAT gateway per AZ — a single NAT gateway is itself a single point of failure.

Step 2 — Entry point and app tier

- Users don’t connect to individual servers. They connect to a **load balancer** in the public subnets (Application Load Balancer — **next week’s topic**).
- The **app servers** (EC2) run in the **private app subnets of both AZs** — no public IPs.
- Each server has an **instance profile** (IAM role) for the AWS access it needs — e.g. read from one S3 prefix.
- Servers are built the same way every time: same AMI, same user data.

Step 3 — Data tier and static files

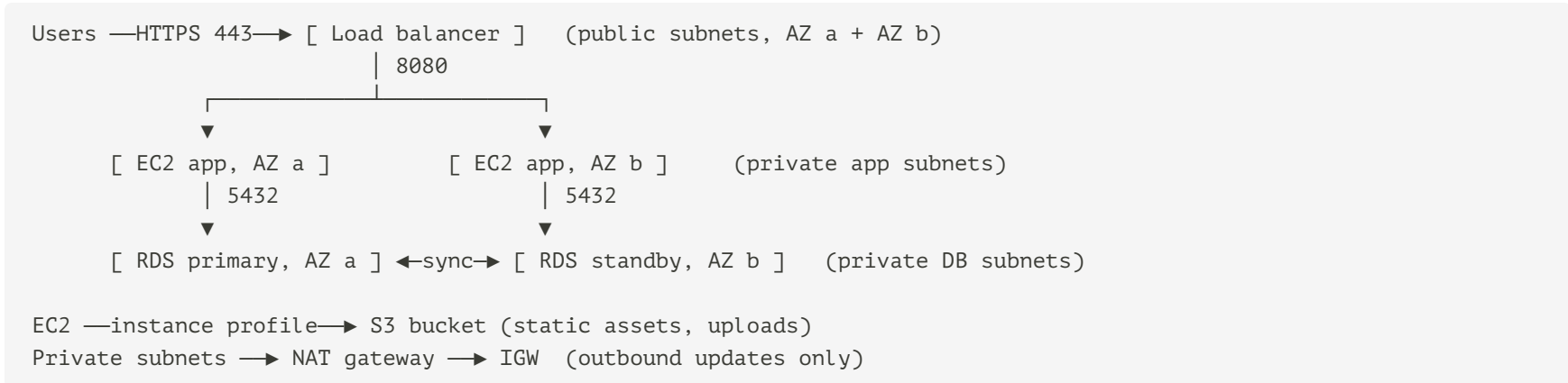
- **RDS** in a DB subnet group across both AZs, **Multi-AZ** on, not publicly accessible, encrypted, automated backups on.
- **S3** for static assets and uploads — Block Public Access on; serve through the app or presigned URLs.
- **EBS** only for what must live on a server's disk; snapshots for backup.

Step 4 — Security groups, chained

Security group	Inbound rule	Source
SG-LB	443 (HTTPS), 80 → redirect	`0.0.0.0/0`
SG-APP	app port (e.g. 8080)	**SG-LB**
SG-DB	5432 / 3306	**SG-APP**

- Each tier only trusts the tier in front of it.
- No SSH from the internet: use Session Manager or a bastion if you must reach a private instance.
- Network ACLs stay at the default unless you need an extra subnet-level deny.

The finished picture



Every box is something you have already built in a lab — except the load balancer, which is next week.

Block 4

Single points of failure

Find the single point of failure

Component	If it's alone…	How to remove the SPOF
One EC2 instance	Instance fails → site down	≥ 2 instances in 2 AZs behind a load balancer
One AZ	AZ outage → everything down	Every tier in ≥ 2 AZs
Single-AZ RDS	DB host or AZ fails → no data	RDS Multi-AZ
One NAT gateway	Its AZ fails → private subnets lose outbound	One NAT gateway per AZ
Data on one EBS volume	Volume lost → data lost	Snapshots; data in RDS/S3

Quick check

AZ `a` goes down in the finished picture. Walk through what happens to a user's request.

The load balancer stops sending traffic to the app server in AZ a and uses AZ b. RDS fails over to the standby in AZ b (same endpoint, a short interruption). Users see at most a brief blip — if every tier really is in both AZs.

Design review checklist

- ☐ Every tier in **two or more AZs**?
- ☐ Only the entry point in **public** subnets?
- ☐ SGs **chained** tier to tier, DB never open to `0.0.0.0/0`?
- ☐ **IAM roles** instead of access keys on servers?
- ☐ **Backups** on, and a restore you've actually tried?
- ☐ **Encryption** at rest for EBS, RDS and S3?
- ☐ Cost: anything running that doesn't need to?

Block 5

Today’s practice

Today’s practice — design and defend

In pairs, no AWS needed:

1. Read the scenario: a university course-registration app
2. Draw the AWS architecture (diagrams.net or paper)
3. Fill a design table: component → pillar → failure it handles
4. Answer the instructor’s **failure walk-through** live

What we check, orally:

- Two AZs for every tier
- Public vs private subnets, with routes
- Chained security groups
- Where the data lives and how it’s backed up
- What happens when *this* fails

Stretch: estimate the monthly cost in the AWS Pricing Calculator — no login needed.

By the end of this lesson, you should be able to

- ☐ Name the six Well-Architected pillars and give an example of each
- ☐ Read an AWS architecture diagram: region, VPC, AZs, subnets, services, flows
- ☐ Draw a three-tier architecture across two AZs from known components
- ☐ Find single points of failure in a design and remove them

Before next lecture

- ☐ Redraw today’s three-tier diagram from memory, with ports on the arrows
- ☐ Note every place in your design where you wrote "load balancer" — that’s next week

Next week fills in the box we left as a placeholder: the load balancer, and how to add and remove servers automatically.

Next lecture

Load Balancing and Auto Scaling — the entry point and the self-healing fleet behind it.